CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

7. Power Laws and Scale-Free Graphs

Lecture 17: Configuration Model

Many real world graphs have small world behaviour (high clustering and short paths) and also a power law degree distribution, due to the presence of nodes of exceptionally high degree. Random graphs do have short paths a but not high clustering; their degree distribution is not described by a power law. The small world graphs in the Watts-Strogatz model have high clustering and short paths, but again, not a power law degree distribution.

So how can one generate a model of a network that does have a power law degree distribution?

Here, we start with a power law, and build a random graph whose degrees are distributed exactly by this power law: quite amazingly, the Configuration Model allows the generation of a random graph for any prescribed degree sequence.

In the next lecture, we'll see a more dynamic approach to this question.

In [1]:
```
import random
from collections import Counter
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
opts = { "with_labels": True, "node_color": 'y' }
```

The Configuration Model

- In principle, a random graph can be generated in such a way that it has any prescribed degree sequence.

Definition: Configuration Model.

- Choose numbers d_i, $i \in X$, so that $\sum d_i = 2m$ is an even number.
- Then regard each degree d_i as d_i **stubs** (half-edges) attached to node i.
- Compute a random **matching** of pairs of stubs and build a graph on X with those (full) edges.

Example. Suppose that $X = \{0, 1, 2, 3, 4\}$ and that we want those nodes to have degrees $d_0 = 3$, $d_1 = 2$ and $d_2 = d_3 = d_4 = 1$.

This gives a list of stubs $(0, 0, 0, 1, 1, 2, 3, 4)$ where each node i appears as often as its degree d_i requires.

A random shuffle of that list is $(0, 2, 3, 0, 1, 0, 4, 1)$.

One way to construct a matching is to simply cut this list in half and match entries of the first half with corresponsing entries in the second half.

$$\begin{matrix} 0 & 2 & 3 & 0 \\ 1 & 0 & 4 & 1 \end{matrix}$$

Note that $\sum d_i = 8 = 2m$ yields $m = 4$ edges $0 - 1$, $2 - 0$, $3 - 4$, and $0 - 1$ ...

A Quick Implementation

In [2]:
```
degrees = [3, 2, 1, 1, 1]
```

- Recall that, in Python, list addresses start at 0, and `networkx` default node names do likewise. Let's adopt this convention here.
- Now entry 3 in position 0 of the list `degrees` stands for 3 entries 0 in the list of stubs, to be constructed. Entry 2 in position 1 stands for 2 entries 1 in the list of stubs and so on. In general, entry d in position i stands for d entries i in the list of stubs.
- Python's list arithmetic (using `m * a` for `m` *repetitions* of a list `a` and `a + b` for the *concatenation* of lists `a` and `b`) can be used to quickly convert a degree sequence into a list of stubs as follows.

In [3]:
```
stubs = [d * [i] for (i, d) in enumerate(degrees)]
stubs
```

Out[3]:
```
[[0, 0, 0], [1, 1], [2], [3], [4]]
```

In [4]:
```
stubs = sum(stubs, [])
stubs
```

Out[4]:
```
[0, 0, 0, 1, 1, 2, 3, 4]
```

- Let's call this process the **stubs list** of a list of integers and wrap it into a `python` function

In [5]:
```
def stubs_list(a):
    return sum([d * [i] for (i, d) in enumerate(a)], [])
```

In [6]:
```
stubs_list(degrees)
```

Out[6]:
```
[0, 0, 0, 1, 1, 2, 3, 4]
```

- How to randomly shuffle this list? The wikipedia page on random permutations

(https://en.wikipedia.org/wiki/Random_permutation#Knuth_shuffles) recommends a simple algorithm for shuffling the elements of a list `a` in place: loop over the positions k of the entries in the list, swapping `a[k]` with `a[j]` for some $j \geq k$ (where possibly $j = k$ and the swap has no visible effect).

In [7]:
```
def knuth_shuffle(a):
    l = len(a)
    for k in range(l):
        j = random.randrange(k, l)
        a[j], a[k] = a[k], a[j]
```

In [8]:
```
a = [1,2,3]
knuth_shuffle(a)
a
```

Out[8]:
```
[1, 3, 2]
```

- Let's test whether this shuffle produces uniformly random outcomes by applying it large number of times to a short list, while keeping track of which permutation occurs how often ...

In [9]:
```
shuffles = {}
for i in range(10000):
    a = [1,2,3]
    knuth_shuffle(a)
    key = tuple(a)
    shuffles[key] = shuffles.get(key, 0) + 1

print(shuffles)
```

```
{(1, 3, 2): 1654, (3, 1, 2): 1660, (2, 1, 3): 1712, (2, 3, 1): 1653,
(3, 2, 1): 1676, (1, 2, 3): 1645}
```

- So it looks like each possible outcome is approximately equally likely.

- Python's `random` module already contains a function `shuffle` which does exactly this.

In [10]:
```
a = [1,2,3]
random.shuffle(a)
a
```

Out[10]:
```
[3, 1, 2]
```

- Let's test whether this `shuffle` produces uniformly random outcomes ...

In [11]:
```
shuffles = {}
for i in range(10000):
    a = [1,2,3]
    random.shuffle(a)
    key = tuple(a)
    shuffles[key] = shuffles.get(key, 0) + 1

print(shuffles)
```

```
{(3, 1, 2): 1708, (1, 2, 3): 1719, (2, 1, 3): 1640, (2, 3, 1): 1625,
(1, 3, 2): 1676, (3, 2, 1): 1632}
```

- Again, equally likely outcomes.

- So we shuffle the stubs ...

In [12]:
```
random.shuffle(stubs)
stubs
```

Out[12]:
```
[4, 1, 2, 0, 3, 0, 1, 0]
```

- Then we match pairs, by cutting the list of stubs into halves and transposing the resulting array of 2 rows ...

In [13]:
```
m = len(stubs) // 2
```

In [14]:
```
edges = [stubs[:m], stubs[m:]]
edges
```

Out[14]:
```
[[4, 1, 2, 0], [3, 0, 1, 0]]
```

In [15]:
```
edges = list(zip(*edges))
edges
```

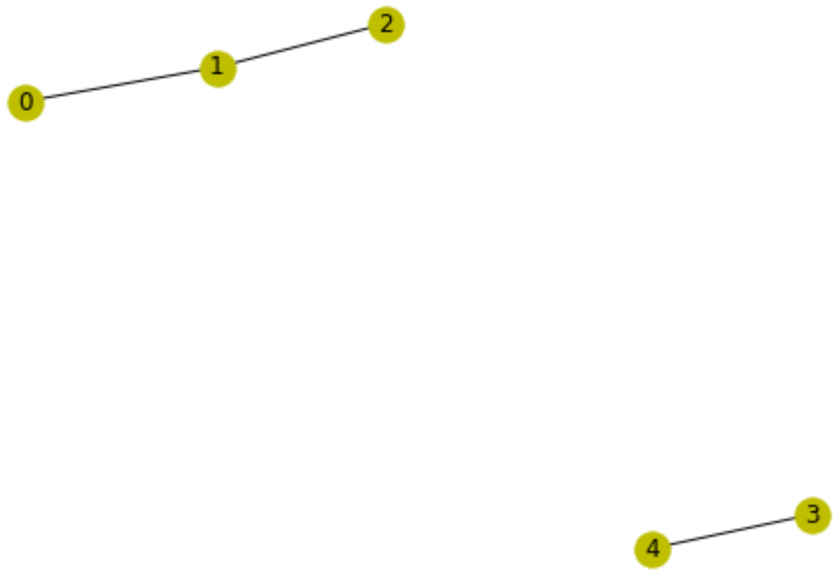
Out[15]:
```
[(4, 3), (1, 0), (2, 1), (0, 0)]
```

In [16]:
```
G = nx.Graph(edges)
```

In [17]:
```
G.number_of_edges()
```

Out[17]:
```
4
```

In [18]:
```
nx.draw(G, **opts)
```



In [19]:
```
G.edges()
```

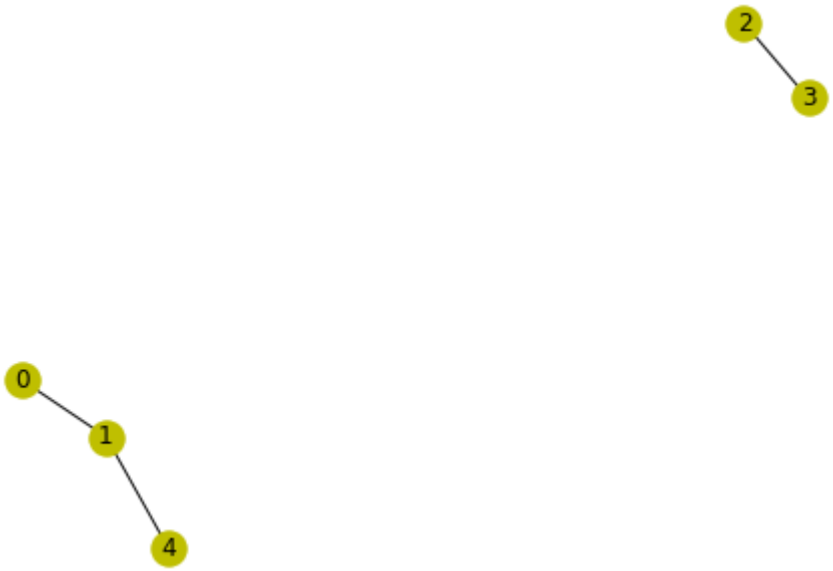
Out[19]:
```
EdgeView([(4, 3), (1, 0), (1, 2), (0, 0)])
```

- All in all, a configuration model can be built as follows.

In [20]:
```
def configuration(degrees):
    m = sum(degrees) // 2  # should check if sum(degrees) is even ...
    stubs = stubs_list(degrees)
    random.shuffle(stubs)
    edges = list(zip(stubs[:m], stubs[m:]))
    return nx.Graph(edges)
```

In [21]:
```
G = configuration([3,2,1,1,1])
nx.draw(G, **opts)
print(G.edges())
```

```
[(2, 3), (1, 4), (1, 0), (0, 0)]
```



Power Law Degree Distribution

A variant of the `powerlaw` from last time can generate the stubs directly.

In [22]:
```
def powerstubs(m, p):

    # distribute 2*m according to a power law
    l = 0
    x = [l]
    for i in range(2*m-1):
        if random.random() < p:
            l += 1
            x.append(l)
        else:
            k = random.choice(x)
            x.append(k)

    return x
```

Let's create 400 stubs for 200 edges and approximately 200 nodes (with $p = \frac{1}{2}$).

In [23]:
```
p = powerstubs(200, 0.5)
print(p)
```

```
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 2, 2, 0, 0, 0, 3, 4, 5, 4, 6, 7,
2, 3, 8, 9, 10, 4, 4, 0, 11, 12, 13, 8, 8, 4, 4, 2, 0, 14, 13, 15, 8,
16, 0, 0, 4, 17, 18, 19, 20, 21, 22, 23, 23, 13, 0, 3, 24, 2, 0, 25, 1
5, 26, 27, 4, 28, 0, 29, 30, 2, 4, 0, 6, 31, 32, 33, 34, 0, 35, 36, 3,
11, 37, 11, 23, 38, 39, 40, 25, 41, 9, 22, 42, 43, 2, 13, 13, 44, 45,
11, 46, 0, 36, 4, 36, 12, 0, 30, 47, 48, 49, 13, 50, 2, 51, 2, 0, 25,
52, 0, 53, 4, 54, 55, 33, 56, 6, 0, 57, 0, 58, 18, 59, 60, 0, 13, 61,
4, 62, 63, 64, 65, 66, 67, 0, 36, 48, 62, 68, 69, 0, 20, 70, 71, 22, 6
0, 1, 48, 9, 0, 7, 0, 72, 28, 73, 18, 74, 75, 8, 76, 77, 0, 0, 14, 78,
79, 12, 0, 80, 81, 82, 83, 63, 84, 85, 4, 86, 87, 2, 37, 13, 2, 88, 1
4, 0, 89, 90, 23, 91, 0, 92, 0, 93, 85, 60, 94, 20, 33, 2, 95, 0, 67,
0, 96, 97, 0, 89, 98, 99, 100, 101, 102, 103, 8, 4, 104, 53, 0, 105, 7
1, 106, 25, 107, 0, 90, 42, 108, 76, 0, 109, 2, 110, 111, 0, 83, 112,
113, 114, 4, 0, 23, 85, 115, 116, 0, 117, 118, 119, 2, 79, 120, 79, 12
1, 25, 122, 0, 123, 18, 124, 125, 13, 126, 43, 13, 4, 45, 0, 127, 128,
58, 129, 130, 122, 131, 47, 132, 133, 134, 45, 135, 136, 20, 3, 18, 13
7, 4, 138, 139, 0, 140, 78, 18, 72, 141, 142, 56, 143, 144, 145, 56,
0, 30, 146, 147, 41, 0, 148, 115, 86, 33, 0, 149, 150, 151, 0, 0, 152,
153, 154, 155, 12, 156, 157, 150, 158, 159, 94, 139, 4, 61, 11, 58, 16
0, 4, 138, 161, 52, 162, 163, 164, 165, 166, 131, 88, 12, 167, 27, 16
8, 1, 33, 86, 169, 12, 1, 170, 171, 172, 173, 145, 174, 175, 176, 177,
101, 178, 179, 2, 58, 180, 181, 6, 182, 39, 183, 184, 185, 12, 186, 2,
187, 188, 189, 190, 85, 0, 4, 191, 67]
```

Use Python's `collections.Counter` to count how often each node occurs, i.e., to determine the node degrees.

In [24]:
```
k = Counter(p)
print(k)
```

```
Counter({0: 58, 4: 20, 2: 16, 13: 10, 12: 7, 8: 6, 18: 6, 3: 5, 11: 5,
23: 5, 25: 5, 33: 5, 1: 4, 6: 4, 20: 4, 36: 4, 58: 4, 85: 4, 9: 3, 14:
3, 22: 3, 30: 3, 45: 3, 48: 3, 56: 3, 60: 3, 67: 3, 79: 3, 86: 3, 7:
2, 15: 2, 27: 2, 28: 2, 37: 2, 39: 2, 41: 2, 42: 2, 43: 2, 47: 2, 52:
2, 53: 2, 61: 2, 62: 2, 63: 2, 71: 2, 72: 2, 76: 2, 78: 2, 83: 2, 88:
2, 89: 2, 90: 2, 94: 2, 101: 2, 115: 2, 122: 2, 131: 2, 138: 2, 139:
2, 145: 2, 150: 2, 5: 1, 10: 1, 16: 1, 17: 1, 19: 1, 21: 1, 24: 1, 26:
1, 29: 1, 31: 1, 32: 1, 34: 1, 35: 1, 38: 1, 40: 1, 44: 1, 46: 1, 49:
1, 50: 1, 51: 1, 54: 1, 55: 1, 57: 1, 59: 1, 64: 1, 65: 1, 66: 1, 68:
1, 69: 1, 70: 1, 73: 1, 74: 1, 75: 1, 77: 1, 80: 1, 81: 1, 82: 1, 84:
1, 87: 1, 91: 1, 92: 1, 93: 1, 95: 1, 96: 1, 97: 1, 98: 1, 99: 1, 100:
1, 102: 1, 103: 1, 104: 1, 105: 1, 106: 1, 107: 1, 108: 1, 109: 1, 11
0: 1, 111: 1, 112: 1, 113: 1, 114: 1, 116: 1, 117: 1, 118: 1, 119: 1,
120: 1, 121: 1, 123: 1, 124: 1, 125: 1, 126: 1, 127: 1, 128: 1, 129:
1, 130: 1, 132: 1, 133: 1, 134: 1, 135: 1, 136: 1, 137: 1, 140: 1, 14
1: 1, 142: 1, 143: 1, 144: 1, 146: 1, 147: 1, 148: 1, 149: 1, 151: 1,
152: 1, 153: 1, 154: 1, 155: 1, 156: 1, 157: 1, 158: 1, 159: 1, 160:
1, 161: 1, 162: 1, 163: 1, 164: 1, 165: 1, 166: 1, 167: 1, 168: 1, 16
9: 1, 170: 1, 171: 1, 172: 1, 173: 1, 174: 1, 175: 1, 176: 1, 177: 1,
178: 1, 179: 1, 180: 1, 181: 1, 182: 1, 183: 1, 184: 1, 185: 1, 186:
1, 187: 1, 188: 1, 189: 1, 190: 1, 191: 1})
```

Using another counter on the degrees yields the degree distribution $p_k = n_k/n$.

In [25]:
```
nk = Counter(k.values())
print(nk)
```

```
Counter({1: 131, 2: 32, 3: 11, 4: 6, 5: 5, 6: 2, 58: 1, 16: 1, 20: 1,
7: 1, 10: 1})
```

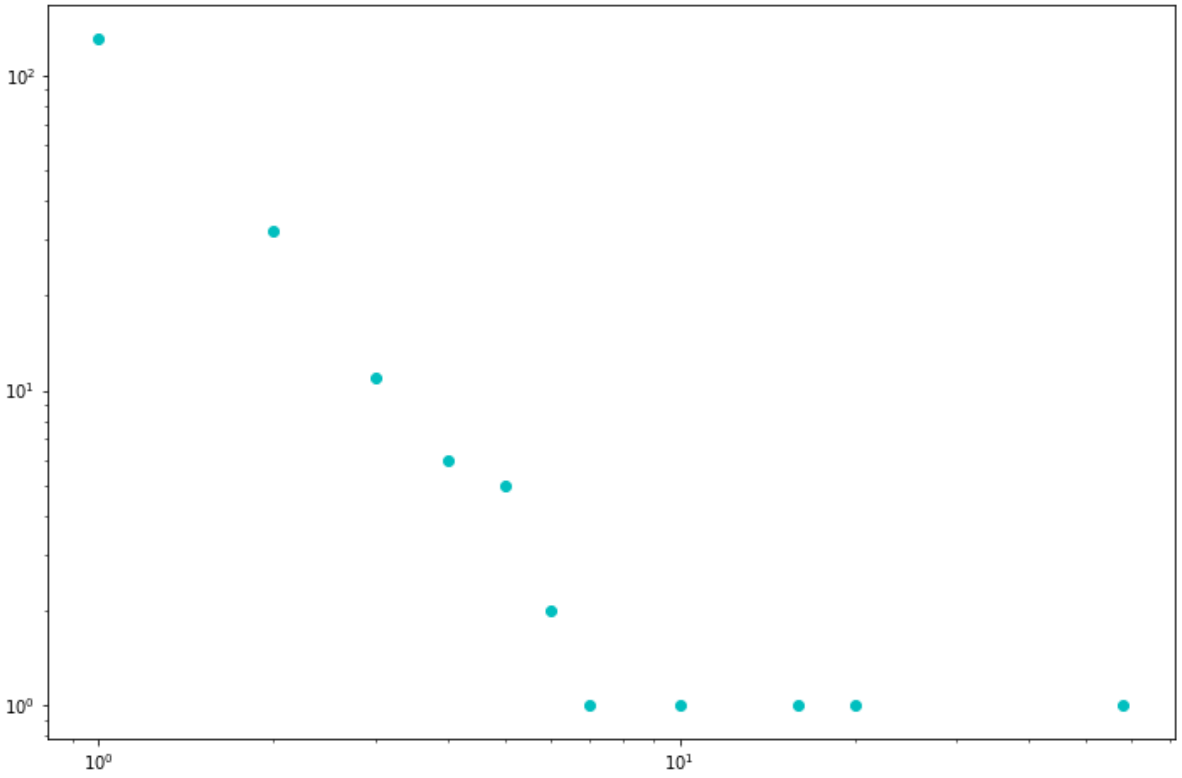
Let's check whether the n_k follow a power law - they really should, by construction.

In [26]:
```
xy = np.array(list(nk.items())).T
print(xy)
```

```
[[ 58   4  16   5  20   1   2   6   3   7  10]
 [  1   6   1   5   1 131  32   2  11   1   1]]
```

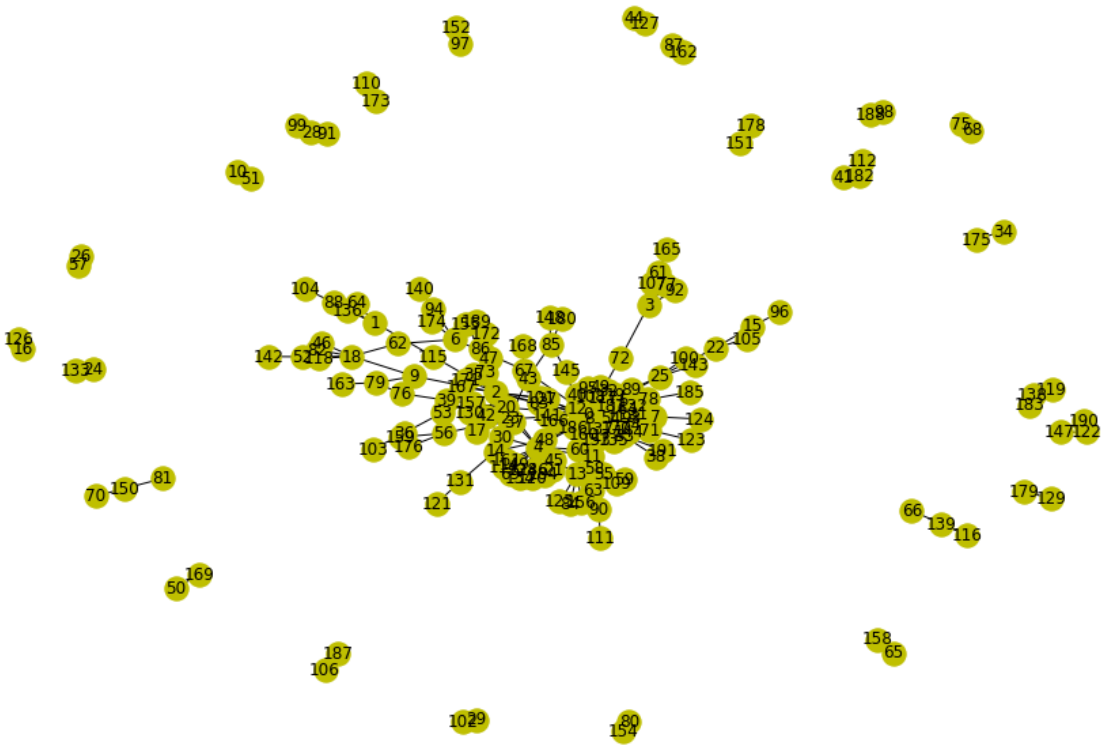
In [27]:
```
plt.figure(figsize=(12,8))
plt.loglog(*xy, 'oc')
```

Out[27]:
```
[<matplotlib.lines.Line2D at 0x7f2e0230c9d0>]
```



Finally, let's construct a graph as a configuration model with the given list of degrees.

In [28]:
```
G = configuration(list(k.values()))
plt.figure(figsize=(12,8))
nx.draw(G, **opts)
```



Maybe it's better to focus on the giant component.

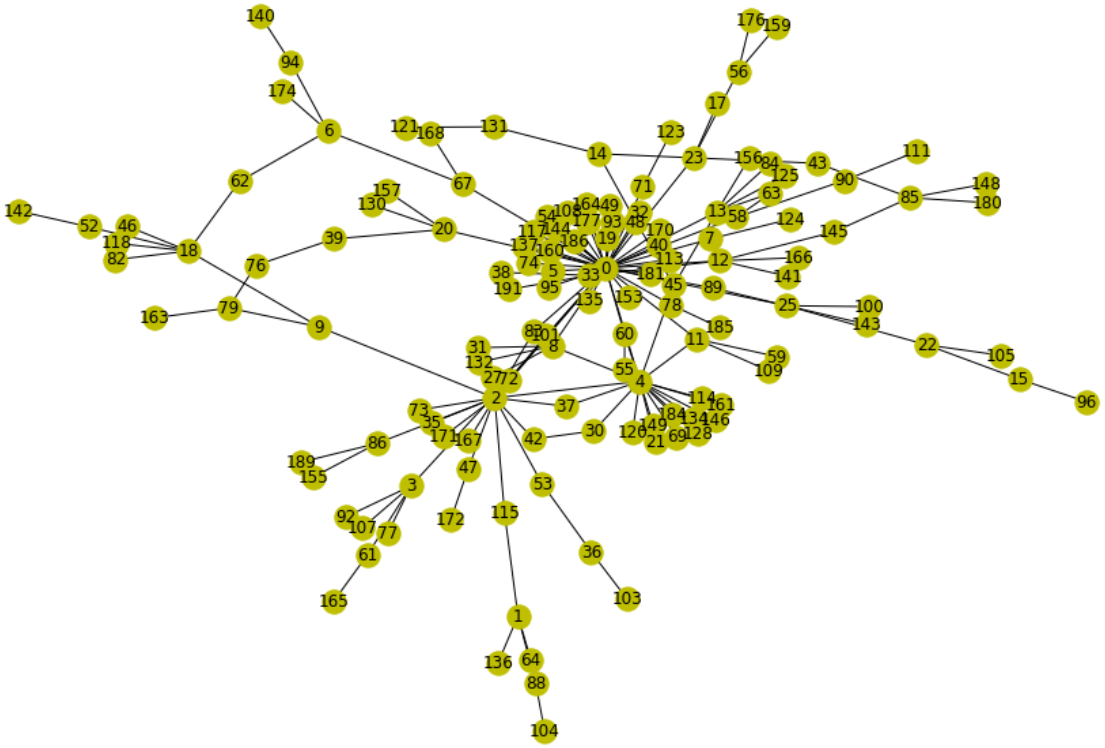
In [29]:
```
print([len(c) for c in nx.connected_components(G)])
```

```
[138, 3, 2, 2, 3, 3, 2, 2, 2, 2, 2, 2, 2, 3, 2, 3, 2, 2, 3, 2, 2, 2,
2, 2, 2]
```

In [30]:
```
print(list(nx.connected_components(G)))
```

```
[{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 17, 18, 19, 20, 2
1, 22, 23, 25, 27, 30, 31, 32, 33, 35, 36, 37, 38, 39, 40, 42, 43, 45,
46, 47, 48, 49, 52, 53, 54, 55, 56, 58, 59, 60, 61, 62, 63, 64, 67, 6
9, 71, 72, 73, 74, 76, 77, 78, 79, 82, 83, 84, 85, 86, 88, 89, 90, 92,
93, 94, 95, 96, 100, 101, 103, 104, 105, 107, 108, 109, 111, 113, 114,
115, 117, 118, 120, 121, 123, 124, 125, 128, 130, 131, 132, 134, 135,
136, 137, 140, 141, 142, 143, 144, 145, 146, 148, 149, 153, 155, 156,
157, 159, 160, 161, 163, 164, 165, 166, 167, 168, 170, 171, 172, 174,
176, 177, 180, 181, 184, 185, 186, 189, 191}, {122, 147, 190}, {169, 5
0}, {178, 151}, {112, 41, 182}, {91, 99, 28}, {16, 126}, {34, 175}, {1
62, 87}, {129, 179}, {106, 187}, {98, 188}, {152, 97}, {138, 119, 18
3}, {29, 102}, {81, 150, 70}, {80, 154}, {173, 110}, {66, 139, 116},
{57, 26}, {24, 133}, {65, 158}, {10, 51}, {75, 68}, {44, 127}]
```

In [31]:
```
C = G.subgraph(max(nx.connected_components(G), key=len))
plt.figure(figsize=(12,8))
nx.draw(C, **opts)
```

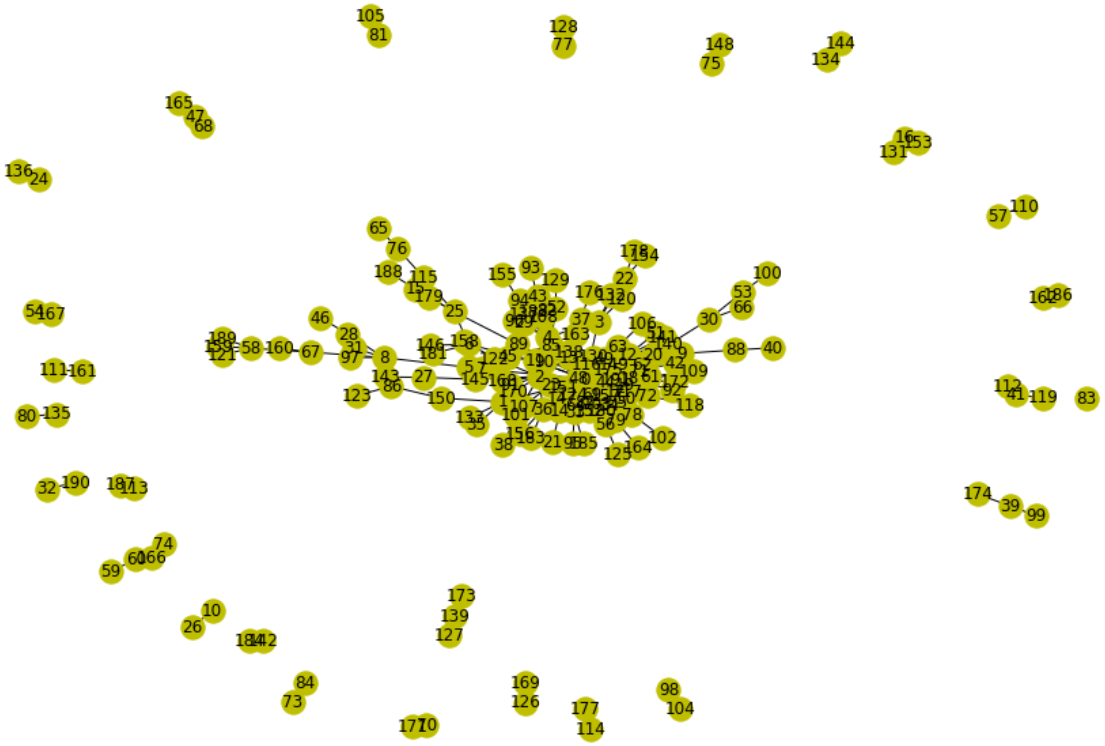


In [32]:
```
print(nx.degree_histogram(C))
```

```
[0, 83, 27, 10, 8, 5, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1]
```

In `networkx`, configuration models can be generated with the function `nx.configuration_model`.

In [33]:
```
G = nx.configuration_model(list(k.values()))
plt.figure(figsize=(12,8))
nx.draw(G, **opts)
```



Code Corner

collections

- `Counter` : [doc] (https://docs.python.org/3/library/collections.html#collections.Counter)

random

- `random` : [doc] (https://docs.python.org/3/library/random.html#random.shuffle)
- `randrange` : [doc] (https://docs.python.org/3/library/random.html#random.randrange)
- `choice` : [doc] (https://docs.python.org/3/library/random.html#random.choice)
- `shuffle` : [doc] (https://docs.python.org/3/library/random.html#random.shuffle)

networkx

- `configuration_model` : [doc] (https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.de

Exercises

1. In your own words, describe and justify how `stubs_list` and `Counter` are mutually inverse processes. How exactly can you recover the stubs from a `Counter` object? How can you recover the stubs from the degree distribution?
2. Experiment with different values for `m` and `p` in the `powerstubs` function. In your own words, how does the resulting graph generated as a random configuration model from a power law degree distribution look different from a random graph in the ER-model $G(r, p)$?
3. Generate some configuration models with power law degree distributions for various choices of γ between 2 and 3. Compute their transitivity, clustering and average shortest pathlengths. Do these values indicate small world behaviour?

In [ ]: